**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ**

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА**

Факультет електроніки та комп'ютерних технологій

# ЗВІТ

## З ЛАБОЛАТОРНОЇ РОБОТИ №3

з дисципліни **"Функційне Програмування"**

на тему **"Нездатні до змін структури даних (immutability) у Python"**

Варіант 1

Виконала:
Студентка 2 курсу
групи ФЕП-23
Бойчук Марія

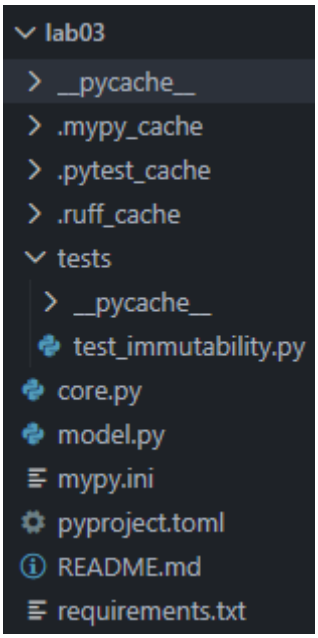
Перевірив:
Професор Свелеба С.

Львів-2026

**Мета:** зрозуміти та застосувати tuple, frozenset, dataclasses(frozen=True), MappingProxyType; замінити мутації на повернення нових копій (pure updates); виміряти ефект (стабільність тестів, час виконання) після переходу на незмінні структури.

## Хід роботи

1. Було створено папку lab03 з файлами для реалізації моделей, основних функцій, тестування та налаштування проєкту:




2. У файлі model.py створено класи Item та Order. Для них використано frozen dataclass, що забороняє змінювати поля створених об'єктів. Також використано tuple, frozenset та MappingProxyType:

model.py > ...

```python
from collections.abc import Mapping
from dataclasses import dataclass, replace
from types import MappingProxyType


@dataclass(frozen=True, slots=True)
class Item:
    sku: str
    price: float
    qty: int


@dataclass(frozen=True, slots=True)
class Order:
    id: int
    paid: bool
    items: tuple[Item, ...]
    tags: frozenset[str]
    meta: Mapping[str, str]

    def subtotal(self) -> float:
        return sum(item.price * item.qty for item in self.items)

    def add_tag(self, tag: str) -> "Order":
        return replace(self, tags=self.tags | frozenset({tag}))

    def with_item(self, item: Item) -> "Order":
        return replace(self, items=self.items + (item,))

    def change_paid(self, paid: bool) -> "Order":
        return replace(self, paid=paid)
```

```
    def __hash__(self) -> int:
        return hash((self.id, self.paid, self.items, self.tags))


def make_meta(**kwargs: str) -> Mapping[str, str]:
    return MappingProxyType(dict(kwargs))


def create_order() -> Order:
    items = (
        Item("A1", 50.0, 2),
        Item("B2", 20.0, 1),
    )

    return Order(
        id=10,
        paid=True,
        items=items,
        tags=frozenset({"delhi", "priority"}),
        meta=make_meta(source="fb_ads", campaign="summer"),
    )
```

3. У файлі core.py реалізовано функції додавання товару, додавання тегу, зміни статусу оплати та обчислення загальної суми замовлення. Функції створюють нові об'єкти, не змінюючи початкові:

core.py > ...

```python
from model import Item, Order

def add_item(
    order: Order,
    sku: str,
    price: float,
    qty: int,
) -> Order:
    item = Item(sku, price, qty)
    return order.with_item(item)

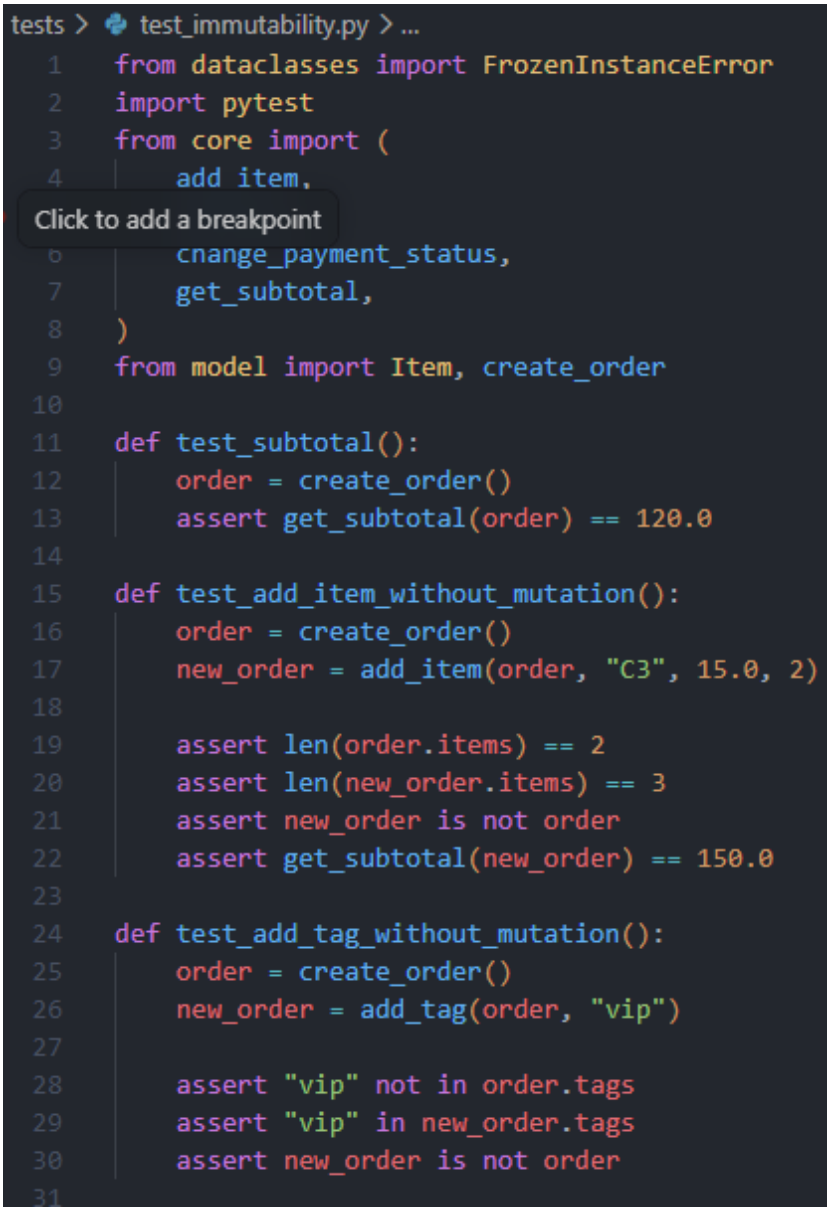
def add_tag(order: Order, tag: str) -> Order:
    return order.add_tag(tag)

def change_payment_status(order: Order, paid: bool) -> Order:
    return order.change_paid(paid)

def get_subtotal(order: Order) -> float:
    return order.subtotal()

```

4. У файлі test_immutability.py створено автоматизовані тести для перевірки правильності обчислень, незмінності об'єктів, захисту метаданих та можливості хешування:



```python
def test_change_payment_status():
    order = create_order()
    new_order = change_payment_status(order, False)

    assert order.paid is True
    assert new_order.paid is False
    assert new_order is not order

def test_order_is_immutable():
    order = create_order()

    with pytest.raises(FrozenInstanceError):
        order.paid = False

def test_mapping_is_read_only():
    order = create_order()

    with pytest.raises(TypeError):
        order.meta["source"] = "changed"

def test_hashability():
    order = create_order()
    new_order = add_tag(order, "vip")

    orders = {order, new_order}

    assert len(orders) == 2
    assert isinstance(hash(order), int)
```

```python
def test_item_is_immutable():
    item = Item("A1", 50.0, 2)

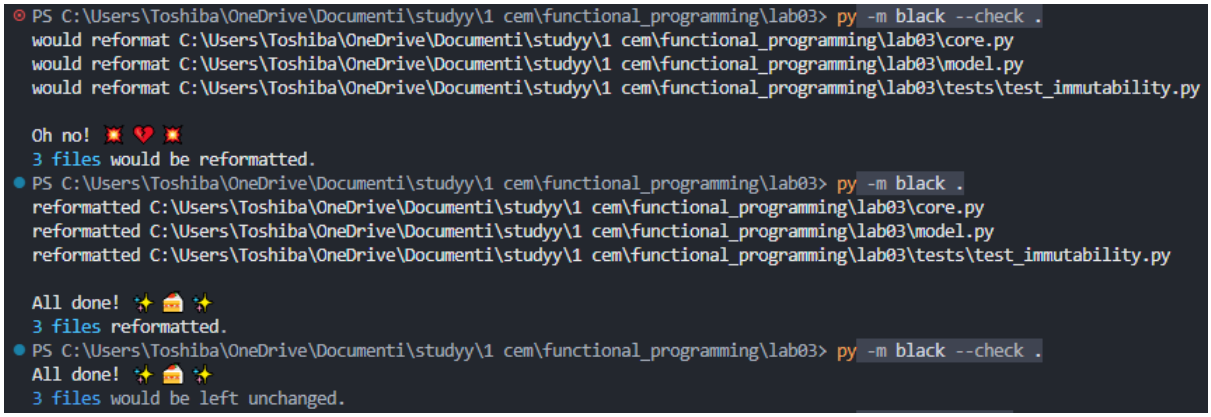
    with pytest.raises(FrozenInstanceError):
        item.price = 100.0

```

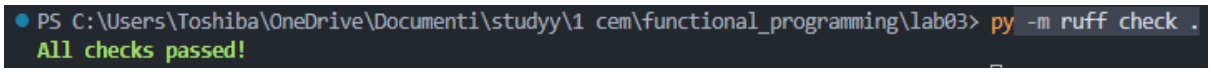
5. Для перевірки правильності роботи програми виконано команду pytest. Усі автоматизовані тести пройшли успішно:

```
PS C:\Users\Toshiba\OneDrive\Documenti\studyy\1 cem\functional_programming\lab03> py -m pytest -v
platform win32 -- Python 3.14.2, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Toshiba\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Toshiba\OneDrive\Documenti\studyy\1 cem\functional_programming\lab03
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.13.0
collected 8 items
tests/test_immutability.py::test_subtotal PASSED                                    [ 12%]
tests/test_immutability.py::test_add_tag_without_mutation PASSED                    [ 37%]
tests/test_immutability.py::test_change_payment_status PASSED                       [ 50%]
tests/test_immutability.py::test_order_is_immutable PASSED                          [ 62%]
tests/test_immutability.py::test_mapping_is_read_only PASSED                        [ 75%]
tests/test_immutability.py::test_hashability PASSED                                 [ 87%]
tests/test_immutability.py::test_item_is_immutable PASSED                           [100%]
============================================================ 8 passed in 0.97s ============================================================
```

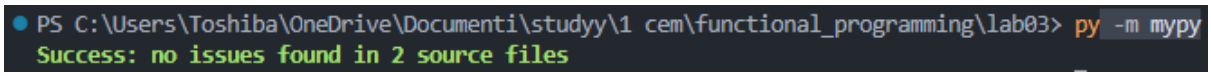
6. За допомогою Black перевірено форматування файлів проєкту. Після автоматичного форматування повторна перевірка пройшла успішно:



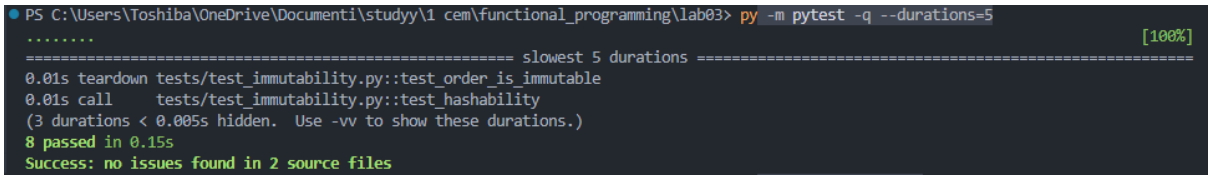

7. За допомогою Ruff перевірено стиль написання коду та правильність імпортів:




8. За допомогою mypy перевірено правильність використання типів даних у програмі:




9. Для вимірювання часу виконання тестів використано команду pytest --durations=5. Вона дозволяє переглянути час виконання найповільніших тестів:




**Висновок:** Під час роботи освоєно незмінні структури даних Python. Реалізовано чисті функції оновлення та перевірено їхню коректність за допомогою тестів.